

THE UNIVERSITY OF BAMENDA

NATIONAL HIGHER
POLYTECHNIC
INSTITUTE



CENTER OF
CYBERSECURITY
AND
MATHEMATICAL
CRYPTOLOGY

UNCONDITIONAL CRYPTOGRAPHY

Homomorphic encryption, functional encryption and attribute based
encryption

By:

GNINTEZEMO MAMEKEM ROSLY

REGISTRATION NUMBER: UBA23EP020

Prof. Fouotsa Emmanuel

(Program coordinator Centre For Cybersecurity And Mathematical Cryptology)

June, 2025

Contents

1	Homomorphic Encryption	2
1.1	Definition	2
1.2	Properties	2
1.3	Example: Paillier Cryptosystem	3
1.4	Verification of Additive Homomorphism	3
1.5	Numerical Example	3
2	Functional Encryption	5
2.1	Definition of Functional Encryption	5
2.2	Properties of Functional Encryption	5
2.3	Security Notions	5
2.4	Example: Inner Product Functional Encryption	6
2.4.1	Setup	6
2.4.2	Verification of Properties	6
3	Attribute-Based Encryption	7
3.1	Definition of Attribute-Based Encryption	7
3.2	Properties of Attribute-Based Encryption	7
3.3	Security Notions	7
3.4	Example: Ciphertext-Policy ABE (CP-ABE)	8
3.4.1	Setup	8
3.4.2	Example Scenario	8
3.5	Numerical Example	9

Unconditional Cryptography

1 Homomorphic Encryption

Homomorphic encryption is a cryptographic technique that allows computations to be performed on encrypted data without decrypting it first. The results of these computations, when decrypted, match the results of the same computations performed on the plaintext. This property makes homomorphic encryption a powerful tool for secure data processing in applications such as cloud computing and privacy-preserving machine learning.

1.1 Definition

Homomorphic encryption is a form of encryption that permits specific algebraic operations on ciphertexts to produce an encrypted result that, when decrypted, corresponds to the result of the same operations applied to the plaintexts. Formally, let E be an encryption function, D be the corresponding decryption function, and m_1, m_2 be plaintexts. A homomorphic encryption scheme satisfies:

$$D(E(m_1) \circ E(m_2)) = m_1 * m_2$$

where $\circ$ is an operation on ciphertexts (e.g., multiplication or addition), and $*$ is the corresponding operation on plaintexts.

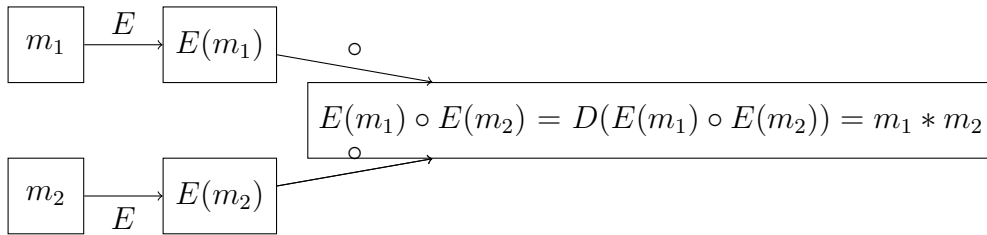


Figure 1: Illustration of homomorphic encryption: Operations on ciphertexts produce encrypted results that decrypt to the plaintext operation.

1.2 Properties

Homomorphic encryption schemes exhibit the following key properties:

- **Additive Homomorphism:** If the encryption scheme allows addition on ciphertexts, it is additively homomorphic. That is, $E(m_1 + m_2) = E(m_1) + E(m_2)$.

- **Multiplicative Homomorphism:** If the scheme allows multiplication on ciphertexts, it is multiplicatively homomorphic. That is, $E(m_1 \cdot m_2) = E(m_1) \cdot E(m_2)$.
- **Security:** The encryption scheme must be semantically secure, meaning that ciphertexts reveal no information about the plaintext without the decryption key.
- **Functionality:** The scheme must support the intended operations (addition, multiplication, or both) while preserving correctness.

Fully homomorphic encryption (FHE) supports both addition and multiplication, enabling arbitrary computations, while partially homomorphic encryption supports either addition or multiplication. Somewhat homomorphic encryption supports a limited number of operations.

1.3 Example: Paillier Cryptosystem

The Paillier cryptosystem is a well-known example of a partially homomorphic encryption scheme that supports additive homomorphism. Below, we describe the Paillier scheme and demonstrate that it satisfies the additive homomorphism property.

The Paillier cryptosystem is based on the difficulty of the composite residuosity problem. The key components are:

- **Key Generation:** Choose two large prime numbers p and q , compute $n = p \cdot q$, and $\lambda = \text{lcm}(p-1, q-1)$. Select a random integer $g \in \mathbb{Z}_{n^2}^*$ such that $\gcd(L(g^\lambda \bmod n^2), n) = 1$, where $L(x) = \frac{x-1}{n}$. The public key is (n, g) , and the private key is λ .
- **Encryption:** For a plaintext $m \in \mathbb{Z}_n$, select a random $r \in \mathbb{Z}_n^*$. The ciphertext is $c = g^m \cdot r^n \bmod n^2$.
- **Decryption:** For a ciphertext $c \in \mathbb{Z}_{n^2}^*$, compute $m = L(c^\lambda \bmod n^2) \cdot L(g^\lambda \bmod n^2)^{-1} \bmod n$.

1.4 Verification of Additive Homomorphism

To show that Paillier is additively homomorphic, consider two plaintexts $m_1, m_2 \in \mathbb{Z}_n$ with ciphertexts $c_1 = g^{m_1} \cdot r_1^n \bmod n^2$ and $c_2 = g^{m_2} \cdot r_2^n \bmod n^2$. Compute the product of the ciphertexts:

$$c_1 \cdot c_2 = (g^{m_1} \cdot r_1^n) \cdot (g^{m_2} \cdot r_2^n) = g^{m_1+m_2} \cdot (r_1 \cdot r_2)^n \bmod n^2$$

This is the encryption of $m_1 + m_2$ with random $r = r_1 \cdot r_2$. Thus, decrypting $c_1 \cdot c_2$ yields $m_1 + m_2 \bmod n$, satisfying the additive homomorphism property:

$$D(c_1 \cdot c_2) = m_1 + m_2 \bmod n$$

1.5 Numerical Example

Let's consider a small example with $p = 3$, $q = 5$, so $n = p \cdot q = 15$, $n^2 = 225$. Compute $\lambda = \text{lcm}(p-1, q-1) = \text{lcm}(2, 4) = 4$.]

Choose $g = 16$ (satisfying the Paillier conditions). The public key is $(n = 15, g = 16)$, and the private key is $\lambda = 4$.

Encrypt two plaintexts $m_1 = 5$ and $m_2 = 7$ with random $r_1 = 2, r_2 = 3$:

- For $m_1 = 5$: $c_1 = g^{m_1} \cdot r_1^n = 16^5 \cdot 2^{15} \mod 225 = 1 \cdot 128 \mod 225 = 128$.
- For $m_2 = 7$: $c_2 = g^{m_2} \cdot r_2^n = 16^7 \cdot 3^{15} \mod 225 = 1 \cdot 27 \mod 225 = 27$.

Compute the product of ciphertexts:

$$c_1 \cdot c_2 = 128 \cdot 27 \mod 225 = 120.$$

Decrypting $c = 120$ using the private key yields $m_1 + m_2 = 5 + 7 = 12 \mod 15$, confirming the additive homomorphism.

2 Functional Encryption

Functional encryption (FE) is an advanced cryptographic paradigm that extends traditional encryption by allowing fine-grained access control over encrypted data. Unlike classical encryption, where decryption is an all-or-nothing operation, FE enables users to compute specific functions on encrypted data without revealing the underlying plaintext. This document provides a formal definition of functional encryption, outlines its key properties, and presents an example to illustrate how these properties are satisfied.

2.1 Definition of Functional Encryption

[Functional Encryption] A functional encryption scheme for a function class $\mathcal{F}$ over a message space $\mathcal{M}$ consists of four probabilistic polynomial-time algorithms:

- **Setup**(1^λ): Takes a security parameter λ and outputs a public key PK and a master secret key MSK.
- **KeyGen**(MSK, f): Takes the master secret key MSK and a function $f \in \mathcal{F}$, and outputs a functional decryption key SK_f .
- **Encrypt**(PK, m): Takes the public key PK and a message $m \in \mathcal{M}$, and outputs a ciphertext CT.
- **Decrypt**(SK_f , CT): Takes a functional decryption key SK_f and a ciphertext CT, and outputs $f(m)$ if CT encrypts m , or $\perp$ otherwise.

The correctness requirement ensures that for any $m \in \mathcal{M}$ and $f \in \mathcal{F}$, if $(\text{PK}, \text{MSK}) \leftarrow \text{Setup}(1^\lambda)$, $\text{SK}_f \leftarrow \text{KeyGen}(\text{MSK}, f)$, and $\text{CT} \leftarrow \text{Encrypt}(\text{PK}, m)$, then $\text{Decrypt}(\text{SK}_f, \text{CT}) = f(m)$ with overwhelming probability.

2.2 Properties of Functional Encryption

Functional encryption schemes are designed to satisfy the following properties:

1. **Functionality**: The scheme correctly computes the function $f(m)$ when decrypting a ciphertext with the corresponding functional key SK_f .
2. **Security**: The scheme ensures that a user with a functional key SK_f learns only $f(m)$ and nothing else about the plaintext m . Formally, this is often defined via *indistinguishability-based security* or *simulation-based security*.
3. **Collusion Resistance**: Even if multiple users collude and combine their functional keys $\{\text{SK}_{f_1}, \dots, \text{SK}_{f_k}\}$, they cannot learn more than what is revealed by $\{f_1(m), \dots, f_k(m)\}$.

2.3 Security Notions

The security of FE is typically formalized as follows:

- **Indistinguishability under Chosen-Plaintext Attack (IND-CPA)**: An adversary with access to functional keys for functions $f_1, \dots, f_k$ cannot distinguish between encryptions of two messages m_0 and m_1 if $f_i(m_0) = f_i(m_1)$ for all i .

- **Simulation-Based Security:** A simulator can replicate the adversary's view using only $\{f_i(m)\}$ and public information, ensuring no additional information about m is leaked.

2.4 Example: Inner Product Functional Encryption

To demonstrate functional encryption, consider an inner product functional encryption scheme, where the function class $\mathcal{F}$ consists of inner product functions over vectors in $\mathbb{Z}_q^n$ for some prime q .

2.4.1 Setup

Let λ be the security parameter, and choose a prime $q = \text{poly}(\lambda)$. The message space is $\mathcal{M} = \mathbb{Z}_q^n \times \mathbb{Z}_q^n$, and the function class is $\mathcal{F} = \{f_{\vec{y}} : \vec{x} \mapsto \langle \vec{x}, \vec{y} \rangle \mid \vec{y} \in \mathbb{Z}_q^n\}$, where $\langle \vec{x}, \vec{y} \rangle = \sum_{i=1}^n x_i y_i \pmod{q}$.

- **Setup**(1^λ): Generate a bilinear group $(\mathbb{G}_1, \mathbb{G}_2, \mathbb{G}_T, e, g_1, g_2, q)$, where $e : \mathbb{G}_1 \times \mathbb{G}_2 \rightarrow \mathbb{G}_T$ is a bilinear map. Choose random $\vec{a} \in \mathbb{Z}_q^n$ and compute $\text{PK} = (g_1, g_1^{\vec{a}}, \mathbb{G}_1, \mathbb{G}_2, \mathbb{G}_T, e)$, $\text{MSK} = \vec{a}$.
- **KeyGen**($\text{MSK}, \vec{y}$): For a vector $\vec{y} \in \mathbb{Z}_q^n$, compute $\text{SK}_{\vec{y}} = \langle \vec{a}, \vec{y} \rangle \in \mathbb{Z}_q$.
- **Encrypt**($\text{PK}, \vec{x}$): For a message $\vec{x} \in \mathbb{Z}_q^n$, choose random $s \in \mathbb{Z}_q$ and output $\text{CT} = (g_1^s, (g_1^{\vec{a}})^s \cdot g_1^{\vec{x}})$.
- **Decrypt**($\text{SK}_{\vec{y}}, \text{CT}$): Given $\text{CT} = (c_1, c_2)$ and $\text{SK}_{\vec{y}} = \langle \vec{a}, \vec{y} \rangle$, compute $e(c_2, g_2^{\langle \vec{a}, \vec{y} \rangle}) / e(c_1, g_2^{\langle \vec{a}, \vec{y} \rangle \cdot \vec{y}}) = e(g_1, g_2)^{\langle \vec{x}, \vec{y} \rangle}$.

2.4.2 Verification of Properties

- **Functionality:** The decryption algorithm computes $e(g_1, g_2)^{\langle \vec{x}, \vec{y} \rangle}$, which correctly reveals the inner product $\langle \vec{x}, \vec{y} \rangle$ (assuming discrete log is hard in $\mathbb{G}_T$).
- **Security:** Under the Decisional Bilinear Diffie-Hellman assumption, the scheme is IND-CPA secure. The adversary learns only $\langle \vec{x}, \vec{y} \rangle$ and cannot distinguish encryptions of $\vec{x}_0$ and $\vec{x}_1$ if $\langle \vec{x}_0, \vec{y} \rangle = \langle \vec{x}_1, \vec{y} \rangle$.
- **Collusion Resistance:** If adversaries collude with keys $\text{SK}_{\vec{y}_1}, \dots, \text{SK}_{\vec{y}_k}$, they learn $\{\langle \vec{x}, \vec{y}_i \rangle\}_{i=1}^k$. Without $\vec{a}$, they cannot compute additional information about $\vec{x}$.

3 Attribute-Based Encryption

Attribute-Based Encryption (ABE) is a cryptographic primitive that enables fine-grained access control over encrypted data based on attributes and policies. Unlike traditional public-key encryption, where a ciphertext is decryptable only by a specific recipient, ABE allows decryption based on whether a user's attributes satisfy a predefined access policy. This document formally defines ABE, outlines its key properties, and provides an example to illustrate how these properties are satisfied.

3.1 Definition of Attribute-Based Encryption

An Attribute-Based Encryption scheme for a universe of attributes $\mathcal{U}$ and an access structure $\mathbb{A}$ over $\mathcal{U}$ consists of four probabilistic polynomial-time algorithms:

- **Setup**(1^λ): Takes a security parameter λ and outputs a public key PK and a master secret key MSK.
- **KeyGen**(MSK, S): Takes the master secret key MSK and a set of attributes $S \subseteq \mathcal{U}$, and outputs a secret key SK_S .
- **Encrypt**(PK, m , A): Takes the public key PK, a message $m \in \mathcal{M}$, and an access structure $A \in \mathbb{A}$, and outputs a ciphertext CT.
- **Decrypt**(SK_S , CT): Takes a secret key SK_S for attribute set S and a ciphertext CT associated with access structure A , and outputs m if S satisfies A , or $\perp$ otherwise.

The correctness requirement ensures that for any message $m \in \mathcal{M}$, access structure $A \in \mathbb{A}$, and attribute set $S \subseteq \mathcal{U}$ satisfying A , if $(PK, MSK) \leftarrow \text{Setup}(1^\lambda)$, $SK_S \leftarrow \text{KeyGen}(MSK, S)$, and $CT \leftarrow \text{Encrypt}(PK, m, A)$, then $\text{Decrypt}(SK_S, CT) = m$ with overwhelming probability.

3.2 Properties of Attribute-Based Encryption

ABE schemes are designed to satisfy the following properties:

1. **Access Control**: The scheme ensures that only users whose attributes satisfy the access structure A can decrypt the ciphertext and recover the message m .
2. **Security**: The scheme guarantees that users whose attributes do not satisfy the access structure learn nothing about the plaintext m . This is typically formalized via *selective-set security* or *full security* under chosen-plaintext attacks (CPA).
3. **Collusion Resistance**: Even if multiple users with different attribute sets $\{S_1, \dots, S_k\}$ collude, they cannot combine their secret keys to decrypt a ciphertext unless at least one of their attribute sets individually satisfies the access structure A .

3.3 Security Notions

The security of ABE is typically formalized as follows:

- **Selective-Set Security**: The adversary declares the target access structure A^* at the beginning of the security game and cannot distinguish between encryptions of

two messages m_0 and m_1 unless they possess a key for an attribute set S satisfying A^* .

- **Full Security:** The adversary can adaptively choose the target access structure A^* after receiving secret keys, and the security guarantee holds as long as none of the provided keys satisfy A^* .

3.4 Example: Ciphertext-Policy ABE (CP-ABE)

To demonstrate ABE, consider a Ciphertext-Policy Attribute-Based Encryption (CP-ABE) scheme, where the access structure is embedded in the ciphertext, and secret keys are associated with attribute sets. We use a simple access policy based on a monotone access tree.

3.4.1 Setup

Let λ be the security parameter, and choose a bilinear group $(\mathbb{G}_1, \mathbb{G}_2, \mathbb{G}_T, e, g_1, g_2, q)$ of prime order $q = \text{poly}(\lambda)$. The attribute universe is $\mathcal{U} = \{a_1, a_2, \dots, a_n\}$, and the access structure is a tree with AND and OR gates.

- **Setup**(1^λ): Choose a bilinear group and random $\alpha, \beta \in \mathbb{Z}_q$. Set $\text{PK} = (g_1, g_1^\beta, e(g_1, g_2)^\alpha, \mathbb{G}_1, \mathbb{G}_2, \mathbb{G}_T, e)$, $\text{MSK} = (\beta, g_2^\alpha)$.
- **KeyGen**(MSK, S): For an attribute set $S \subseteq \mathcal{U}$, choose random $r \in \mathbb{Z}_q$ and compute $\text{SK}_S = (g_2^\alpha \cdot (g_2^\beta)^r, g_1^r, \{g_1^{r/h_i}\}_{a_i \in S})$, where $h_i \in \mathbb{Z}_q$ are attribute-specific secrets included in PK.
- **Encrypt**(PK, m, A): For a message $m \in \mathbb{G}_T$ and access tree A , choose random $s \in \mathbb{Z}_q$ and construct $\text{CT} = (A, C = m \cdot e(g_1, g_2)^{\alpha s}, C' = g_1^s, \{C_i = (g_1^{h_i})^s\}_{a_i \in \text{leaves of } A})$.
- **Decrypt**(SK_S, CT): If S satisfies A , use the access tree to reconstruct $e(g_1, g_2)^{\beta s}$ via pairings, then compute $m = C / (e(C', g_2^\alpha \cdot (g_2^\beta)^r) / e(g_1^r, e(g_1, g_2)^{\beta s}))$.

3.4.2 Example Scenario

Suppose the access policy is $A = (a_1 \text{ AND } a_2) \text{ OR } a_3$. A user with attributes $S = \{a_1, a_2\}$ can decrypt, as can a user with $S = \{a_3\}$. A user with $S = \{a_1\}$ cannot decrypt.

Verification of Properties

- **Access Control:** Only users whose attributes satisfy the access tree A can reconstruct $e(g_1, g_2)^{\beta s}$ and recover m . For example, $S = \{a_1, a_2\}$ satisfies $(a_1 \text{ AND } a_2)$, allowing decryption.
- **Security:** Under the Decisional Bilinear Diffie-Hellman assumption, the scheme is selective-set secure. An adversary with keys for attribute sets not satisfying A cannot distinguish encryptions of m_0 and m_1 .
- **Collusion Resistance:** If two users with $S_1 = \{a_1\}$ and $S_2 = \{a_2\}$ collude, they cannot decrypt a ciphertext with policy $(a_1 \text{ AND } a_2)$, as their keys use independent randomness r_1, r_2 , preventing reconstruction of the required pairing.

3.5 Numerical Example

Consider a CP-ABE scheme with $q = 17$, attribute universe $\mathcal{U} = \{a_1, a_2, a_3\}$, and access policy $A = (a_1 \text{ AND } a_2) \text{ OR } a_3$. Let the bilinear group be $(\mathbb{G}_1, \mathbb{G}_2, \mathbb{G}_T, e, g_1, g_2, 17)$.

Setup: Choose $\alpha = 4, \beta = 6 \in \mathbb{Z}_{17}$, and attribute secrets $h_1 = 2, h_2 = 3, h_3 = 5$. Set $\text{PK} = (g_1, g_1^6, e(g_1, g_2)^4, h_1 = 2, h_2 = 3, h_3 = 5)$, $\text{MSK} = (6, g_2^4)$.

KeyGen: For $S = \{a_1, a_2\}$, choose $r = 7$. Compute $\text{SK}_S = (g_2^4 \cdot (g_2^6)^7, g_1^7, \{g_1^{7/2}, g_1^{7/3}\})$.

Encrypt: For message $m \in \mathbb{G}_T$ and policy A , choose $s = 9$. Compute $\text{CT} = (A, C = m \cdot e(g_1, g_2)^{4 \cdot 9}, C' = g_1^9, \{C_1 = (g_1^2)^9, C_2 = (g_1^3)^9, C_3 = (g_1^5)^9\})$.

Decrypt: Since $S = \{a_1, a_2\}$ satisfies $(a_1 \text{ AND } a_2)$, use C_1, C_2 , and the key components to reconstruct $e(g_1, g_2)^{6 \cdot 9}$ via pairings, then compute $m = C / e(g_1, g_2)^{4 \cdot 9}$. This confirms the access control property, as $S = \{a_1\}$ would fail to satisfy A and prevent decryption.

References

- [1] P. Paillier, "Public-Key Cryptosystems Based on Composite Degree Residuosity Classes," EUROCRYPT 1999.
- [2] D. Boneh, A. Sahai, and B. Waters, "Functional Encryption: Definitions and Challenges," TCC 2011.
- [3] B. Waters, "Ciphertext-Policy Attribute-Based Encryption: An Expressive, Efficient, and Provably Secure Realization," PKC 2011.
- [4] C. Gentry, "Fully Homomorphic Encryption Using Ideal Lattices," STOC 2009.